① CSci 2041
16 Feb 26

Announcements
- ~~continuing~~ Continuing to record
- Midterm exam 27 Feb 26

"classic" map functions
- leftReduce
- rightReduce

Continuation passing style (CPS)
- Definition
- forLoop
- generateBools

leftReduce f [1; 2; 3] ∅ ⇒ (f (f (f ∅ 1) 2) 3)

Calls to f associate ("bunch up") to the left.

```
let leftReduce func objects default =
    let rec reducing objects =
      match objects
      with [] -> default |
           head :: tail ->
             func (reducing tail) head
    in reducing objects ;;
```

not a tail recursion

rightReduce f [1; 2; 3] ∅ ⇒ (f 1 (f 2 (f 3 ∅)))

Calls to f associate to the right

```
let rightReduce func objects default =
    let rec reducing objects =
      match objects
      with [] -> default |
           head :: tail ->
             func head (reducing tail)
    in reducing objects ;;
```

not a tail recursion.

(2) CSci 2041
16 Feb 26

Functions usually communicate by returning values

$f\ x \Rightarrow y$

Functions can also provide values by calling other functions, called continuations.

$\underbrace{f\ x\ (\text{fun } y \to \ldots)}_{f\text{'s continuation}}$ } we (probably) don't care what f returns

$f\ (g\ x)$? continuation passing form

$g\ x\ (\text{fun } y \to (f\ (\text{fun } z \to \ldots)))$

why do we want to do this?

$f\ x\ (\text{fun } y \to \ldots)$

1. what if we don't know how many values f will provide?
   - ∅ values – never call f's continuation.
   - 1 value – f calls its continuation once.
   - 2 values – f calls its continuation twice.

     ≡

   - n values – f calls its continuation n times.

   continuation passing generalizes function returns.

2. what if n is potentially large?
   can't put all possible values in e.g. a list.
   Want to look at only a few.
   Makes more sense – more efficient – to generate values one at a time.

③ CSci 2041
16 Feb 26

Write a function in CPS that acts like a for loop.
Counting loop.

```
let forLoop etc min max =
  let rec looping index =
    if index <= max
    then (etc index ;
          looping (index+1))
    else (.)
  in looping min ;;
```

continuation
et cetera
my contention

tail recursion

```
forLoop (fun count -> ...) 0 5
```

this expression is executed for count = 0, 1, 2 ..., 5

What does ; do?

$e_1 ; e_2 \ldots ; e_{n-1} ; e_n$

Executes $e_1, e_2 \ldots e_{n-1}$ in order & discards their values. Finally it executes e_n & returns its value.

Type of forLoop:

$$\underbrace{(int \to 'a)}_{etc} \to \underbrace{int}_{min} \to \underbrace{int}_{max} \to \underbrace{unit}_{\text{returned value}}$$

Nested for-loops

```
(forLoop
   (fun i ->
      forLoop
         (fun j -> ...)
         0 5)
   0 5)
```

executed for all $0 \le i \le 5$
$0 \le j \le 5$

If min > max then etc never gets called.

④ CSci 2041
16 Feb 16

suppose we want to generate all possible lists of bool's, 1 list at a time. Lists have length n.

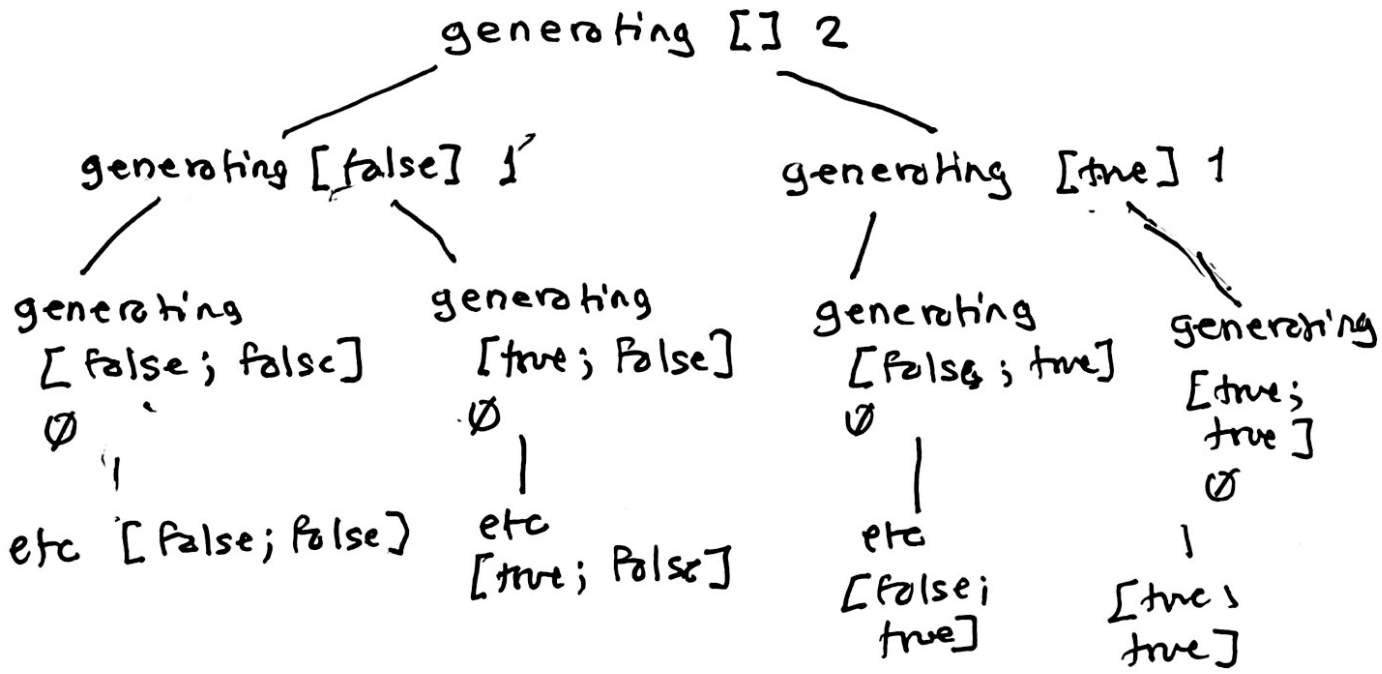
$n = 2$ [false; false]
$n \geq 0$ [false; true]
[true; false]
[true; true]

Looks like part of a truth table.
2^n possible lists
$2^{10} \approx 1000$ $2^{20} \approx 10^6$

```
let generateBools etc n =
  let rec generating bools n =
    match n
    with 0 -> eto bools |
         _ -> generating (false :: bools) (n-1) ;
      is tail — generating (true :: bools) (n-1)
  in generating [] n ;;
```

list of bools
no of elements remaining to be generated
not tail
Choosing the next element of bools in a different way.

Tree of calls to generating



(5) CSci 2011
16 Feb 26

Use CPS to generate combinatorial objects
permutations, combinations, sequences.

CPS function often don't return interesting values. Often don't care what the returned value is.
We care what the continuation gets.

generatingBools — guts of a tautology checker for propositions.

next few lectures.